
Phân công code vấn đáp đồ án

X509Certificate

Phần việc chi tiết của thành viên

Phạm Minh Quân

Xác thực client, Verification Lab & LAN mode

Tài liệu này mô tả chi tiết phần code, lý thuyết X.509 cần nắm, các test case và cách tích hợp cho cụm **xác thực phía client (5 bước verify), Verification Lab và chế độ nộp CSR qua mạng LAN**, gắn trực tiếp với mã nguồn thực tế của đồ án trong `src/core/verify.py`, `src/services/`, `src/legacy/`, `src/infra/` và `scripts/`.

Môn: Mã hoá và Ứng dụng (MHUD)

Học phần đồ án X.509 Certificate

Ngày biên soạn: 14/06/2026

Contents

1	Mục tiêu và phạm vi phần việc	3
1.1	Ảnh xạ khái niệm → file code → vai trò	3
1.2	Ranh giới với các thành viên khác	4
2	Kiến trúc tổng thể và vị trí của thành viên	4
2.1	Ba nguồn cert mà client phải xác thực	4
2.2	Luồng 5 bước xác thực	4
2.3	Kiến trúc LAN mode	5
3	Phần code cần làm (chi tiết)	5
3.1	Orchestrator — <code>verify_certificate_full</code>	5
3.2	Bước 1 — Verify chữ ký bằng Root CA: <code>verify_signature</code>	6
3.3	Bước 3 — Hostname/SAN: <code>check_hostname</code> & wildcard	7
3.4	Bước 4 — CRL: <code>check_crl</code> & <code>_verify_crl_signature</code>	7
3.5	Bước 5 — OCSP: <code>check_ocsp</code>	8
3.6	Lấy cert qua mạng + chống SSRF/DoS	8
3.7	Bước phụ — Pinning / TOFU: <code>check_pinned</code>	9
3.8	Cert ngoài — <code>save_external_cert</code> & <code>parse_cert_summary</code>	9
3.9	Verification Lab — <code>ServerManager</code> (back-end)	10
3.10	Verification Lab — <code>renew_server</code> (rotate key)	11
3.11	Verification Lab — GUI <code>lifecycle_demo</code>	12
3.12	UI upload — <code>upload_external_view</code>	12
3.13	LAN mode — server nhận CSR: <code>submit_remote_csr</code>	12
3.14	LAN mode — HTTP API: <code>csr_api_server</code>	13
3.15	LAN mode — client gửi: <code>remote_csr_client</code>	14
3.16	CLI — <code>submit_csr_lan.py</code> & <code>dump_cert.py</code>	14
4	Cần nắm để vấn đáp	15
5	Test case và demo	16
5.1	Bảng test case của cụm	16
5.2	Đoạn unit test gợi ý	17
5.3	Lệnh chạy demo/test	18
5.4	Kịch bản demo trực tiếp	18
6	Tích hợp vào chương trình chung	18
7	Bổ sung sau rà soát: API LAN đầy đủ & chi tiết khác	19
7.1	LAN không chỉ “nộp CSR” — API customer self-service đầy đủ	19
7.2	Vài chi tiết verify dễ bị hỏi	20

8 Checklist chuẩn bị vấn đáp

20

1 Mục tiêu và phạm vi phân việc

Thành viên **Phạm Minh Quân** phụ trách cụm “**Xác thực client, Verification Lab & LAN mode**”. Đây là phần *phía người dùng cuối*: nhận một chứng chỉ X.509 (từ socket server, từ file upload, hay từ máy khác trên LAN), chạy đủ **5 bước xác thực**, tổng hợp kết quả PASS/FAIL, và cung cấp một “phòng thí nghiệm” (Verification Lab) để dựng đủ các tình huống cert tốt/xấu phục vụ demo. Ngoài ra cụm còn lo chế độ **LAN**: cho phép một máy khách sinh keypair + CSR rồi nộp qua HTTP API tới máy Admin để xin cấp chứng chỉ.

1.1 Ánh xạ khái niệm → file code → vai trò

Khái niệm	File code	Vai trò
5 bước xác thực + orchestrator	src/core/verify.py	verify_certificate_full gọi 5 bước, tính overall=AND; chứa verify_signature, check_validity, check_hostname, check_crl, check_ocsp, check_pinned.
Lấy cert qua mạng + guard	src/core/verify.py	fetch_certificate (socket GET_CERT), _validate_fetch_url (chống SSRF), _fetch_capped (chống DoS).
Upload & lưu cert ngoài	src/services/external_certs.py	save_external_cert (SHA-256 dedup, BOLA), parse_cert_summary.
Verification Lab (back-end)	src/legacy/server_manager.py	ServerManager sinh nhiều socket server theo flavor, tamper, renew.
Verification Lab (GUI)	src/legacy/lifecycle_demo.py	GUI Tkinter, _do_verify chạy 5 bước thread nền, banner PASS/FAIL, launch_as_toplevel.
LAN mode — server nhận CSR	src/services/remote_csr.py	submit_remote_csr (login/auto-register, parse+verify CSR, chèn DB).
LAN mode — client gửi	src/services/remote_csr_client.py	_post_json, submit_csr_to_admin_api, health check.
LAN mode — HTTP API	src/infra/csr_api_server.py	Routing /api/csr/submit, _authorized token, _is_loopback_bind.
UI upload cert ngoài	src/ui/customer/upload_external_view.py	Paste/Browse PEM → lưu → VerifyExternalDialog chạy 5 bước.
CLI tiện ích	scripts/submit_csr_lan.py, scripts/dump_cert.py	Sinh keypair+CSR rồi POST; xuất cert ra PEM để test.

Table 1: Phần việc của Phạm Minh Quân ánh xạ vào mã nguồn thực tế.

Tóm tắt một câu khi mở đầu vấn đáp: “*Em phụ trách toàn bộ phía client — nhận chứng chỉ (qua socket, file upload hoặc LAN) rồi chạy 5 bước xác thực (chữ ký/Root CA, thời hạn, hostname, CRL, OCSP) và tổng hợp PASS/FAIL; đồng thời xây Verification Lab để dựng các cert tốt/xấu cho demo, và chế độ LAN cho máy khác nộp CSR qua HTTP API.*”

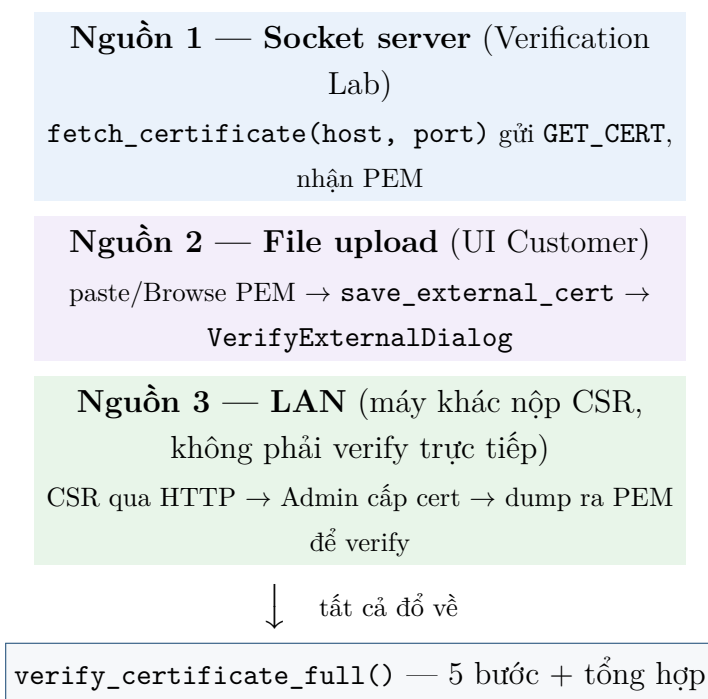
1.2 Ranh giới với các thành viên khác

Cụm này *tiêu thụ* kết quả của các nhóm khác và *điều phối* pipeline. Cụ thể: parse cert (`x509.load_pem_x509_certificate`) thuộc nhóm đọc cert; primitive verify chữ ký (`public_key.verify`) và verify chữ ký CRL là phần mật mã của thành viên khác; còn việc *gọi* chúng đúng thứ tự, gom kết quả, xử lý lỗi và đưa ra kết luận cuối cùng là của Quân.

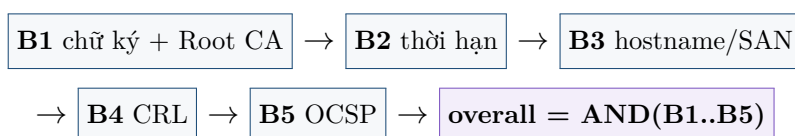
2 Kiến trúc tổng thể và vị trí của thành viên

2.1 Ba nguồn cert mà client phải xác thực

Điểm thống nhất của cả cụm: dù cert đến từ đâu, tất cả đều đổ về một hàm duy nhất là `verify_certificate_full()`. Ba nguồn:



2.2 Luồng 5 bước xác thực

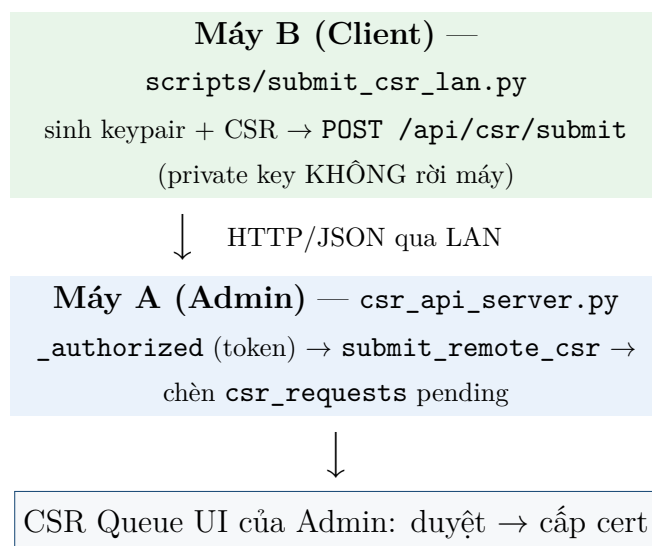


Bước phụ “Pin warning (advisory)” chạy SAU khi tính overall → không ảnh hưởng kết luận chính.

Bước	Hàm	Ý nghĩa
B1	<code>verify_signature</code>	Cert được Root CA trong Trust Store ký hợp lệ?
B2	<code>check_validity</code>	Hiện tại nằm trong [Not Before, Not After]?
B3	<code>check_hostname + _dnsname_matches</code>	Hostname khớp SAN (DNS/wildcard/IP)?
B4	<code>check_crl + _verify_crl_signature</code>	Serial có trong danh sách thu hồi CRL?
B5	<code>check_ocsp</code>	Trạng thái trực tuyến GOOD/REVOKED?
phụ	<code>check_pinned</code>	Fingerprint khớp pin đã lưu (TOFU/rotation)? <i>advisory</i>

Table 2: Sáu hàm cấu thành pipeline xác thực client.

2.3 Kiến trúc LAN mode



3 Phần code cần làm (chi tiết)

3.1 Orchestrator — `verify_certificate_full`

Đây là điểm hội tụ của cả cụm: nó parse cert, nạp Trust Store, gọi 5 bước, gom (name, ok, msg), và tính overall. Bước phụ pin được append *sau* khi tính overall:

```

1 def verify_certificate_full(cert_bytes, hostname, trust_store_dir,
2                             log_callback=None, pin_dir="received_certs",
3                             peer_address=None):
4     """Chạy đủ 5 bước. Trả về (overall_pass, results, cert)."""
5     cert = x509.load_pem_x509_certificate(cert_bytes)
6     trusted_cas = load_trust_store(trust_store_dir)
7     steps = [
8         ("Bước 1 - Verify chữ ký bằng Root CA (Trust Store)",
9          lambda: verify_signature(cert, trusted_cas)),

```

```

10         ("Bước 2 - Thời hạn hiệu lực",          lambda: check_validity(cert)),
11         (f"Bước 3 - Hostname ({hostname})", lambda: check_hostname(cert, hostname)),
12         ("Bước 4 - CRL check ...",              lambda: check_crl(cert, trusted_cas)),
13         ("Bước 5 - OCSP check",                  lambda: check_ocsp(cert)),
14     ]
15     results = []
16     for name, fn in steps:
17         ok, msg = fn()
18         results.append((name, ok, msg))
19     overall = all(ok for _, ok, _ in results)  # KẾT LUẬN = AND tất cả bước chính
20     # ... save_server_cert + check_pinned được append SAU -> không vào overall
21     return overall, results, cert

```

Quyết định thiết kế cần bảo vệ:

- `overall = all(...)`: chỉ HỢP LỆ khi *mọi* bước chính PASS — nguyên tắc “mặc định từ chối”.
- Gom `(name, ok, msg)` thay vì fail-fast: chạy hết để báo *đầy đủ* lý do (ví dụ vừa hết hạn vừa bị thu hồi).
- Tách advisory khỏi `overall`: pin warning chỉ là cảnh báo MITM, không được làm hỏng kết luận khi cert được gia hạn hợp lệ.
- `log_callback`: cho UI in real-time từng bước — demo trực quan.

3.2 Bước 1 — Verify chữ ký bằng Root CA: `verify_signature`

Bước 1 gom 3 việc: kiểm Trust Store, nối issuer→subject, verify chữ ký:

```

1  def verify_signature(cert, trusted_cas):
2      """Bước 1: Verify chữ ký server cert bằng Root CA public key trong Trust
3      ↪ Store."""
4      if not trusted_cas:
5          return False, "Trust Store rỗng - không có Root CA để verify"
6      issuer_ca = _find_trusted_issuer(cert, trusted_cas)  # ca.subject == cert.issuer
7      if issuer_ca is None:
8          return False, ("Không tìm thấy Root CA tin cậy có subject khớp issuer="
9                          f"{cert.issuer.rfc4514_string()}" trong Trust Store")
10     try:
11         _verify_signature_with_pubkey(
12             issuer_ca.public_key(), cert.signature,
13             cert.tbs_certificate_bytes, cert.signature_hash_algorithm)
14         return True, "Chữ ký HỢP LỆ - verified bằng Root CA ..."
15     except InvalidSignature:
16         return False, "Chữ ký KHÔNG hợp lệ - cert bị giả mạo hoặc tampered"

```

Ba trạng thái thoát: (1) Trust Store rỗng → FAIL; (2) không tìm thấy issuer khớp →

FAIL (Root CA không tin cậy); (3) verify ném `InvalidSignature` → FAIL (cert bị sửa). Verify tính trên `tbs_certificate_bytes` (phần “To-Be-Signed”), nên chỉ cần 1 bit trong cert đổi là phát hiện ngay — đây là nguyên lý mà flavor `tampered` chứng minh.

3.3 Bước 3 — Hostname/SAN: `check_hostname` & `wildcard`

Bước 3 lấy SAN (DNS + IP) rồi khớp; với DNS có hỗ trợ wildcard `*.example.com` qua `_dnsname_matches`:

```

1 def check_hostname(cert, hostname):
2     """Bước 3: Hostname phải khớp với SAN."""
3     san = cert.extensions.get_extension_for_oid(
4         ExtensionOID.SUBJECT_ALTERNATIVE_NAME).value
5     dns_names = san.get_values_for_type(x509.DNSName)
6     ip_addresses = [str(ip) for ip in san.get_values_for_type(x509.IPAddress)]
7     try:
8         ipaddress.ip_address(hostname)          # hostname là IP?
9         if hostname in ip_addresses: return True, "KHỚP (IP)"
10    except ValueError:
11        if any(_dnsname_matches(p, hostname) for p in dns_names):
12            return True, "KHỚP (DNS/wildcard)"
13    return False, f"Hostname '{hostname}' KHÔNG khớp với SAN"
14
15 def _dnsname_matches(pattern, hostname):
16     pattern, hostname = pattern.lower().rstrip("."), hostname.lower().rstrip(".")
17     if pattern == hostname: return True
18     if not pattern.startswith("*"): return False
19     suffix = pattern[1:]                      # ".example.com"
20     return hostname.endswith(suffix) and hostname.count(".") == pattern.count(".")

```

Ý chính: wildcard chỉ phủ *một* cấp con — `*.a.com` khớp `x.a.com` nhưng không khớp `y.x.a.com` (kiểm bằng đếm số dấu chấm bằng nhau). IP thì khớp tuyệt đối, không có wildcard.

3.4 Bước 4 — CRL: `check_crl` & `_verify_crl_signature`

```

1 def check_crl(cert, trusted_cas):
2     """Bước 4: Tải CRL, verify chữ ký CRL bằng Root CA, đối chiếu serial."""
3     crl_ext = cert.extensions.get_extension_for_oid(
4         ExtensionOID.CRL_DISTRIBUTION_POINTS)          # ExtensionNotFound -> bỏ qua
5     crl_urls = [gn.value for dp in crl_ext.value if dp.full_name
6                 for gn in dp.full_name
7                 if isinstance(gn, x509.UniformResourceIdentifier)]
8     for url in crl_urls:
9         crl = x509.load_pem_x509_crl(_fetch_capped(url, _MAX_CRL_BYTES, timeout=5))
10        sig_ok, issuer_ca, err = _verify_crl_signature(crl, trusted_cas)
11        if not sig_ok:
12            return False, f"CRL không đáng tin từ {url}: {err}"

```

```

13     entry = crl.get_revoked_certificate_by_serial_number(cert.serial_number)
14     if entry is not None:
15         return False, "Chúng chỉ BỊ THU HỒI theo CRL ..."
16     return True, "Không có trong CRL; CRL signature OK theo Root CA"

```

Trước khi tin CRL phải verify *chữ ký* của chính CRL (CRL cũng do Root CA ký qua `KeyUsage(crl_sign)`), nếu không kẻ tấn công có thể đưa một CRL giả “rỗng”. Sau đó mới đối chiếu `cert.serial_number` với danh sách thu hồi. Không có CRLDP $\Rightarrow$ bỏ qua (PASS) — điểm yếu này được OCSP bù.

3.5 Bước 5 — OCSP: `check_ocsp`

```

1 def check_ocsp(cert):
2     """Bước 5: Gọi OCSP service kiểm tra trạng thái trực tuyến."""
3     aia = cert.extensions.get_extension_for_oid(
4         ExtensionOID.AUTHORITY_INFORMATION_ACCESS) # ExtensionNotFound -> bỏ qua
5     ocsp_urls = [ad.access_location.value for ad in aia.value
6                  if ad.access_method == AuthorityInformationAccessOID.OCSP]
7     for url in ocsp_urls:
8         body = _fetch_capped(f"{url}?serial={cert.serial_number}",
9                             _MAX_OCSP_BYTES, timeout=5)
10        status = json.loads(body.decode()).get("status", "UNKNOWN")
11        if status == "GOOD": return True, f"OCSP status = GOOD (từ {url})"
12        if status == "REVOKED": return False, f"OCSP status = REVOKED (từ {url})"
13        return False, f"OCSP status không xác định: {status}"

```

OCSP hỏi *một* cert theo serial qua URL trong AIA — mới hơn và nhẹ hơn CRL. Scenario `revoked_ocsp_only` minh hoạ: OCSP biết cert đã thu hồi trong khi CRL chưa publish kịp $\Rightarrow$ B5 FAIL còn B4 PASS.

3.6 Lấy cert qua mạng + chống SSRF/DoS

Cert lấy qua socket; còn CRL/OCSP tải từ URL *nằm trong cert* — mà cert là dữ liệu không tin cậy, nên cần hai lớp guard:

```

1 def fetch_certificate(host, port, timeout=5.0):
2     """Kết nối socket, gửi 'GET_CERT', nhận PEM. Trả (cert_bytes, peer_address)."""
3     with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
4         s.settimeout(timeout); s.connect((host, port))
5         peer_ip, peer_port = s.getpeername()[:2] # địa chỉ THỰC đã kết nối
6         s.sendall(b"GET_CERT")
7         return _recv_blob(s), f"{peer_ip}:{peer_port}"
8
9 def _validate_fetch_url(url): # chống SSRF
10    parsed = urlparse(url)
11    if parsed.scheme not in ("http", "https"): # chặn file://, gopher://, ...
12        raise UnsafeURLError(f"Scheme '{parsed.scheme}' không cho phép")

```

```

13     if not parsed.hostname:
14         raise UnsaferURLError("URL không có hostname")
15
16 def _fetch_capped(url, max_bytes, timeout=5.0):          # chống DoS
17     _validate_fetch_url(url)
18     with urllib.request.urlopen(url, timeout=timeout) as resp:
19         data = resp.read(max_bytes + 1)                  # +1 để phát hiện vượt
20     if len(data) > max_bytes:
21         raise UnsaferURLError(f"Response > {max_bytes} bytes - từ chối (DoS)")
22     return data

```

Ý chính: chỉ cho http/https (chặn file:///etc/passwd, gopher://...); cap kích thước (10MB cho CRL, 64KB cho OCSP). `peer_address` ghi lại IP:port *thực* đã kết nối, tách bạch với hostname người dùng nhập (có thể bị DNS đầu độc) — phục vụ audit.

3.7 Bước phụ — Pinning / TOFU: `check_pinned`

Pinning ghi nhớ *fingerprint* SHA-256 của cert để phát hiện cert bị tráo (MITM). Pin store v2 hỗ trợ rotation có overlap:

```

1 def check_pinned(cert, hostname, base_dir="received_certs"):
2     """Pin store v2 - TOFU + rotation overlap. Advisory."""
3     store      = _load_pin_store(pin_path, hostname)    # tự migrate v1 -> v2
4     pruned_fps = _prune_expired_pins_inplace(store, now) # bỏ pin đã hết hạn
5     pins       = store["pins"]
6     if not pins:                                       # (a) TOFU: lần đầu -> lưu
7         store["pins"].append(_build_pin_entry(cert, current_fp, now_iso))
8         return True, "Pin TẠO MỚI (TOFU) ..."
9     for pin in pins:                                   # (b) khớp -> OK
10        if pin["fingerprint_sha256"] == current_fp:
11            return True, "Pin KHỚP ..."
12    same_issuer = [p for p in pins if p.get("issuer") == current_issuer
13                   or not p.get("issuer")]
14    if same_issuer:                                     # (c) cùng issuer -> rotation
15        store["pins"].append(_build_pin_entry(cert, current_fp, now_iso))
16        return True, "Pin ROTATION được chấp nhận ..."
17    return False, "Pin MISMATCH! ... Có thể đổi CA HỢP LỆ hoặc đang bị MITM."

```

Ý chính: (a) TOFU lưu pin lần đầu; (b) khớp → OK; (c) khác fingerprint nhưng cùng issuer → rotation hợp lệ (thêm pin mới, giữ pin cũ làm backup trong giai đoạn overlap); chỉ *khác issuer hoàn toàn* mới trả `ok=False` (ngghi MITM). Pin hết hạn tự bị prune. Đây là advisory vì đổi cert hợp lệ (renew/rotate key) rất bình thường.

3.8 Cert ngoài — `save_external_cert` & `parse_cert_summary`

Cho phép customer upload *bất kỳ* cert X.509 nào để theo dõi + verify. `save_external_cert` parse, tính SHA-256 fingerprint để chống upload trùng, và lưu DB:

```

1 def save_external_cert(uploader_id, data, notes, db_path):
2     """Parse + lưu. Reject nếu cùng uploader đã upload cert có cùng fingerprint."""
3     cert = _parse_cert(data)                                # thử PEM rồi DER
4     pem_bytes = cert.public_bytes(Encoding.PEM)             # chuẩn hoá về 1 format
5     fp = _fingerprint_sha256(cert)                          # SHA-256 của DER
6     with transaction(db_path) as conn:
7         dup = conn.execute(
8             "SELECT id FROM external_certs "
9             "WHERE uploader_id = ? AND fingerprint_sha256 = ?",
10            (uploader_id, fp)).fetchone()
11         if dup:
12             raise ExternalCertError(f"Bạn đã upload cert này trước đó
13             ↪ ({dup['id']}).")
14         cur = conn.execute(
15             "INSERT INTO external_certs "
16             "(uploader_id, cert_pem, fingerprint_sha256, notes, uploaded_at) "
17             "VALUES (?, ?, ?, ?, ?)", (uploader_id, pem_bytes, fp, notes, now))
18     return {...}

```

Hai điểm bảo mật phải nêu: (1) **Dedup theo fingerprint per uploader** — cùng người không upload trùng một cert. (2) **BOLA guard** — `get_external_cert(cert_id, uploader_id, ...)` luôn có điều kiện `WHERE id=? AND uploader_id=?`, nên user A không xem/xóa được cert của user B dù biết `cert_id`. `parse_cert_summary` trả metadata (subject, issuer, serial, SAN, public key) để preview *trước khi* lưu DB.

3.9 Verification Lab — ServerManager (back-end)

ServerManager dựng nhiều socket server đồng thời, mỗi server phục vụ một *flavor*. Mỗi server có 3 trạng thái độc lập:

Trục	Giá trị
lifecycle	valid / expired / revoked
revocation_scope	none / ocsf_only / both
wire_mutation	none / tampered

Table 3: Mô hình 3 trục — flavor là tổ hợp khởi tạo của 3 trục này.

```

1 FLAVOR_TO_STATE = {
2     "valid": ("valid", "none", "none"),
3     "expired": ("expired", "none", "none"),
4     "revoked_both": ("revoked", "both", "none"),
5     "revoked_ocsf_only": ("revoked", "ocsf_only", "none"),
6     "tampered": ("valid", "none", "tampered"),
7 }
8
9 def add_server(self, name, port, flavor):

```

```

10     key = generate_rsa_keypair()
11     cert, serial = create_server_cert_signed_by_ca( # Root CA ký, KHÔNG self-signed
12         server_private_key=key, ca_cert=self.issuer_cert,
13         ca_private_key=self.issuer_key, common_name="localhost",
14         dns_names=["localhost", "127.0.0.1"],
15         ocsp_url=self.ocsp_url, crl_url=self.crl_url, expired=(flavor=="expired"))
16     if flavor == "revoked_ocsp_only": # chỉ OCSP DB
17         revoke_serial_ocsp_only(serial, self.ocsp_db_path)
18     elif flavor == "revoked_both": # OCSP DB + publish CRL ngay
19         revoke_serial_ocsp_only(serial, self.ocsp_db_path)
20         build_and_publish_crl(self.issuer_cert, self.issuer_key, ...)
21     if flavor == "tampered": # lật 1 bit trong chữ ký
22         tampered_pem = tamper_cert_pem(original_pem); ... write ...
23     entry = ServerEntry(name, port, flavor, serial, cert_path, key_path)
24     self._start_socket_server(entry) # background thread

```

Ý chính: mỗi flavor sinh đúng một tình huống xác thực để demo. `tampered` lật 1 bit chữ ký $\Rightarrow$ B1 FAIL; `revoked_ocsp_only` đẩy serial vào OCSP DB nhưng *không* publish CRL $\Rightarrow$ B5 FAIL mà B4 PASS. Socket server trả PEM khi nhận `GET_CERT`:

```

def _handle_conn(self, conn, addr, entry):
    data = conn.recv(1024).decode(errors="ignore").strip()
    if data == "GET_CERT":
        with open(entry.cert_path, "rb") as f: cert_pem = f.read()
        conn.sendall(len(cert_pem).to_bytes(4, "big")) # 4-byte length prefix
        conn.sendall(cert_pem)

```

3.10 Verification Lab — `renew_server` (rotate key)

`renew_server` sinh cert mới (ký lại bởi Root CA), ghi serial cũ vào `previous_serials` để audit, và cho client thấy fingerprint mới ở lần `GET_CERT` kế tiếp — minh họa rotation hợp lệ phía pin store:

```

1  def renew_server(self, name, rotate_key=True, validity_days=365):
2      entry = self.servers.get(name)
3      if entry.wire_mutation == "tampered": raise ValueError(...) # untamper trước
4      if entry.lifecycle == "revoked": raise NotImplementedError(...)
5      old_serial = entry.serial
6      key = generate_rsa_keypair() if rotate_key else load_private_key(entry.key_path)
7      cert, new_serial = create_server_cert_signed_by_ca(..., expired=False)
8      save_cert_and_key(cert, key, entry.cert_path, entry.key_path) # ghi đè
9      entry.previous_serials.append(old_serial) # audit oldest -> newest
10     entry.serial = new_serial
11     entry.lifecycle = "valid" # flavor GIỮ NGUYÊN
12     return entry

```

3.11 Verification Lab — GUI `lifecycle_demo`

GUI Tkinter nhúng vào dashboard admin qua `launch_as_toplevel`. Khi bấm Verify, `_do_verify` chạy trên thread nền để không treo UI, rồi cập nhật banner PASS/FAIL qua `root.after`:

```

1  def _do_verify(self, entry):
2      host = "localhost"
3      cert_bytes, peer_address = fetch_certificate(host, entry.port) # socket
4      overall, results, cert_obj = verify_certificate_full(
5          cert_bytes, host, trust_store_dir=self.trust_store_dir,
6          log_callback=self._thread_log, peer_address=peer_address)
7      for step_name, ok, _ in results:
8          self._thread_log(f"    [{'PASS' if ok else 'FAIL'}] {step_name}")
9      self.root.after(0, self.set_result, "pass" if overall else "fail") # banner
10
11 def launch_as_toplevel(parent):
12     """Mở Lab như Toplevel của app admin đã chạy mainloop."""
13     win = tk.Toplevel(parent); win.transient(parent); App(win); return win

```

Lab dùng *thư mục và port riêng* (`lab/`, OCSP 9888, CRL 9889) để không đụng hạ tầng Prod (8888/8889). Khi đóng, file cert được giữ lại (`cleanup_files=False`) để pin warning ổn định qua các lần mở/đóng GUI.

3.12 UI upload — `upload_external_view`

UI Customer cho paste/Browse PEM, preview rồi lưu; tab “Của tôi” liệt kê và mở `VerifyExternalDialog` chạy 5 bước với Trust Store của hệ thống:

```

1  def _run(self):                                     # trong VerifyExternalDialog
2      hostname = self.host_entry.get().strip()
3      publish_active_to_trust_store(self.app.db_path, TRUST_STORE_DIR) # đảm bảo có
   ↪ Root CA
4      overall, results, _ = verify_certificate_full(
5          bytes(self.cert_rec["cert_pem"]), hostname=hostname,
6          trust_store_dir=TRUST_STORE_DIR, log_callback=self._log,
7          pin_dir="received_certs", peer_address=None)
8      for step_name, ok, _ in results:
9          self._log(f"    [{'PASS' if ok else 'FAIL'}] {step_name}",
10                  "ok" if ok else "fail")
11     self._set_banner("pass" if overall else "fail")

```

3.13 LAN mode — server nhận CSR: `submit_remote_csr`

Máy Admin nhận CSR từ xa, login/auto-register customer, parse + verify chữ ký CSR, rồi chèn vào hàng đợi CSR như một request bình thường:

```

1 def submit_remote_csr(*, username, password, csr_pem, key_name, db_path):
2     """Nhận CSR từ máy khác, tạo csr_requests pending. Private key KHÔNG rồi
   ↪ client."""
3     csr = parse_csr(csr_bytes)
4     if not verify_csr_signature(csr):                # CSR tự ký bằng private key
   ↪ client
5         raise RemoteCSRError("CSR signature is invalid")
6     common_name = _csr_common_name(csr); san_list = _csr_san_list(csr)
7     public_key_pem = csr.public_key().public_bytes(
8         serialization.Encoding.PEM, serialization.PublicFormat.SubjectPublicKeyInfo)
9     user = _login_or_register_customer(username, password, db_path)
10    with transaction(db_path) as conn:
11        cur = conn.execute(                        # schema cần 1 row
   ↪ customer_keys
12        "INSERT INTO customer_keys (owner_id, name, algorithm, key_size, "
13        " public_key_pem, encrypted_private_key, gcm_nonce, created_at) "
14        "VALUES (?, ?, ?, ?, ?, ?, ?, ?)",
15        (user["id"], final_key_name, "RSA", key_size,
16         public_key_pem, b"", b"", now))            # private key RỖNG
17    key_id = cur.lastrowid
18    cur = conn.execute(
19        "INSERT INTO csr_requests (requester_id, customer_key_id, common_name, "
20        " san_list_json, csr_pem, status, submitted_at) "
21        "VALUES (?, ?, ?, ?, ?, 'pending', ?)",
22        (user["id"], key_id, common_name, san_json, csr_bytes, now))

```

Ý chính cốt lõi: `encrypted_private_key=b""` — private key *không bao giờ* rời máy client. Admin chỉ nhận public key (trong CSR) + chữ ký CSR để chứng minh client *sở hữu* private key tương ứng. Đây chính là giá trị của mô hình CSR.

3.14 LAN mode — HTTP API: `csr_api_server`

API HTTP chạy trên máy Admin, routing theo path, bảo vệ bằng token và kiểm tra bind address:

```

1 def _is_loopback_bind(host):
2     host = host.strip().lower()
3     if host in ("localhost", "127.0.0.1", "::1"): return True
4     try: return ipaddress.ip_address(host).is_loopback
5     except ValueError: return False
6
7 def start_csr_api_server(*, db_path, host=..., port=..., token=...,
   ↪ log_callback=None):
8     if not token and not _is_loopback_bind(host):        # ép token khi mở ra LAN
9         raise ValueError("CSR API token is required when binding to a LAN/public
   ↪ address.")
10    class Handler(BaseHTTPRequestHandler):
11        def _authorized(self):

```

```

12         if not token: return True
13         return self.headers.get("X-CSR-API-Token") == token
14     def do_POST(self):
15         if not self._authorized():
16             self._send_json(401, {"ok": False, "error": "unauthorized"}); return
17         payload = self._read_json_payload()          # cap 512KB chống DoS
18         if self.path == "/api/csr/submit":
19             rec = submit_remote_csr(username=..., password=..., csr_pem=...,
19             ↪ db_path=db_path)
20             self._send_json(201, {"ok": True, "csr": rec}); return

```

Ba lớp bảo vệ: (1) `_is_loopback_bind` buộc phải có token nếu bind ra địa chỉ LAN/công khai (chỉ localhost mới được phép không token); (2) `_authorized` kiểm header `X-CSR-API-Token`; (3) `_read_json_payload` cap 512KB chống request khổng lồ. Lỗi nghiệp vụ trả 400, lỗi không lường trả 500.

3.15 LAN mode — client gửi: `remote_csr_client`

Hàm `_post_json` là lõi: serialize JSON, gán token, xử lý mọi lỗi mạng thành `RemoteCSRClientError`, kiểm trường ok:

```

1  def _post_json(*, api_url, path, payload, token="", timeout=10.0):
2      req = urllib.request.Request(
3          api_url.rstrip("/") + path, data=json.dumps(payload).encode("utf-8"),
4          headers={"Content-Type": "application/json"}, method="POST")
5      if token: req.add_header("X-CSR-API-Token", token)
6      try:
7          with urllib.request.urlopen(req, timeout=timeout) as resp:
8              data = json.loads(resp.read().decode("utf-8"))
9      except urllib.error.HTTPError as e:
10         raise RemoteCSRClientError(f"HTTP {e.code}:
10         ↪ {e.read().decode('utf-8', 'replace')}")
11     except urllib.error.URLError as e:
12         raise RemoteCSRClientError(f"Cannot connect to Admin API: {e}")
13     if not data.get("ok"):
14         raise RemoteCSRClientError(str(data.get("error", "remote API failed")))
15     return data

```

3.16 CLI — `submit_csr_lan.py` & `dump_cert.py`

`submit_csr_lan.py` là client CLI hoàn chỉnh: sinh keypair, build CSR, lưu private key *tại chỗ*, rồi POST:

```

1  key = generate_rsa_keypair(args.key_size)
2  csr = build_csr(key, common_name=args.domain, san_list=san_list)
3  csr_pem = csr_to_pem(csr)
4  key_pem = key.private_bytes(serialization.Encoding.PEM,

```

```

5     serialization.PrivateFormat.PKCS8, serialization.NoEncryption())
6 key_path.write_bytes(key_pem); csr_path.write_bytes(csr_pem)    # private key Ủ LẠI
↳  máy B
7 payload = {"username": ..., "password": ..., "key_name": ..., "csr_pem":
↳  csr_pem.decode("ascii")}
8 req = urllib.request.Request(args.server.rstrip("/") + "/api/csr/submit",
9     data=json.dumps(payload).encode("utf-8"),
10    headers={"Content-Type": "application/json"}, method="POST")
11 if args.token: req.add_header("X-CSR-API-Token", args.token)

```

dump_cert.py xuất một cert từ DB ra file PEM để test luồng upload/verify:

```

# python scripts/dump_cert.py          -> list cert; python ... <id> -> dump PEM
row = conn.execute("SELECT cert_pem, common_name, serial_hex "
                    "FROM issued_certs WHERE id = ?", (cert_id,)).fetchone()
Path(out_path).write_bytes(pem if isinstance(pem, bytes) else pem.encode())

```

4 Cần nắm để vấn đáp

Hỏi: 5 bước xác thực từng bước làm gì?

Đáp: B1 verify chữ ký cert bằng public key Root CA trong Trust Store (cert được CA tin cậy ký, không bị sửa). **B2** kiểm thời hạn Not Before/Not After. **B3** hostname khớp SAN (DNS, wildcard 1 cấp, hoặc IP). **B4** tải CRL, verify chữ ký CRL, đối chiếu serial xem có bị thu hồi. **B5** hỏi OCSP trạng thái trực tuyến GOOD/REVOKED. Kết luận overall = AND của cả 5 bước.

Hỏi: Trust Store là gì và vì sao cần?

Đáp: Là tập các Root CA mà client *chủ động* chọn tin cậy. Verify chữ ký chỉ chứng minh “cert do ai đó ký”, chứ không trả lời “ai đó có đáng tin không”. Trust Store cung cấp điểm neo tin cậy: chỉ cert leo tới được một Root CA *trong* Trust Store mới được chấp nhận. Trust Store rỗng $\Rightarrow$ B1 FAIL ngay.

Hỏi: CRL và OCSP khác nhau thế nào? Vì sao cần cả hai?

Đáp: CRL là *danh sách* serial bị thu hồi, client tải về rồi tra — offline được nhưng có thể cũ (publish theo chu kỳ). OCSP hỏi *trực tuyến* trạng thái của *một* cert — mới hơn, nhẹ hơn nhưng cần mạng. Dùng cả hai để bù khuyết: OCSP bắt kịp thu hồi sớm, CRL làm fallback. Scenario `revoked_ocsp_only` cho thấy OCSP biết trước khi CRL kịp publish.

Hỏi: Pinning/TOFU là gì và vì sao chỉ là advisory?

Đáp: TOFU = tin cert thấy lần đầu rồi ghi nhớ fingerprint SHA-256; lần sau cert đổi fingerprint mà *khác issuer* thì cảnh báo (ngghi MITM). Nó là advisory (không tính vào overall) vì đổi cert hợp lệ (renew/rotate key) rất bình thường — nếu để nó làm FAIL thì sẽ báo động giả mỗi lần gia hạn. Store v2 phân biệt rotation (cùng issuer → thêm pin, giữ pin cũ overlap) với mismatch (khác issuer).

Hỏi: Vì sao phải chặn SSRF và giới hạn kích thước khi tải CRL/OCSP?

Đáp: Vì URL của CRL/OCSP *nằm trong cert* — do người phát hành cert quyết định, mà cert có thể độc hại. Không guard thì kẻ tấn công đặt URL kiểu `file:///etc/passwd` (SSRF, đọc file nội bộ) hoặc trỏ tới response khổng lồ (DoS). `_validate_fetch_url` chỉ cho `http/https`; `_fetch_capped` cap size (10MB CRL, 64KB OCSP).

Hỏi: Kiến trúc LAN CSR đảm bảo private key không rời máy client thế nào?

Đáp: Máy client tự sinh keypair và *lưu private key tại chỗ*; chỉ gửi đi CSR — chứa public key và một chữ ký do private key tạo. Server Admin verify chữ ký CSR (`verify_csr_signature`) để chắc chắn client thực sự sở hữu private key, rồi cấp cert cho public key đó. Trong DB, cột `encrypted_private_key` để rỗng (`b""`). Admin không bao giờ thấy private key.

Hỏi: Flavor tampered chứng minh điều gì?

Đáp: Nó lật 1 bit trong cert đã được Root CA ký. Vì chữ ký được tính trên `tbs_certificate_bytes`, chỉ 1 bit đổi là giá trị verify không khớp ⇒ `InvalidSignature` ⇒ B1 FAIL. Điều này chứng minh trực quan rằng *không thể sửa cert sau khi đã ký* mà không bị phát hiện — chính là tính toàn vẹn mà chữ ký số bảo đảm.

Hỏi: Vì sao API LAN ép phải có token khi bind ra địa chỉ không phải loopback?

Đáp: Bind `0.0.0.0` hay IP LAN nghĩa là máy khác trên mạng truy cập được. Nếu không xác thực, ai cũng có thể spam CSR vào hàng đợi Admin. `_is_loopback_bind` cho phép chạy không token *chỉ* khi localhost (an toàn vì chỉ máy mình), còn mở ra LAN thì `start_csr_api_server` raise `ValueError` buộc đặt token, kiểm qua header `X-CSR-API-Token`.

5 Test case và demo

5.1 Bảng test case của cụm

Test case	Cách dựng	Kỳ vọng
valid	Lab: add server flavor valid	overall=True, 5 bước PASS
expired	flavor expired	FAIL B2 (thời hạn)
revoked_both	flavor revoked_both	FAIL B4 (CRL) và B5 (OCSP)
revoked_ocsp_only	flavor revoked_ocsp_only	FAIL B5; B4 PASS
tampered	flavor tampered	FAIL B1 (InvalidSignature)
Trust store rỗng	verify với trusted_cas=[]	FAIL B1 (Trust Store rỗng)
Upload cert ngoài	UI Customer paste/Browse PEM	lưu OK, verify 5 bước
Upload trùng	upload lại cert cũ	ExternalCertError (dedup)
SSRF guard	cert có CRLDP file://...	UnsafeURLError, B4 FAIL
LAN submit CSR	submit_csr_lan.py	csr_requests pending trên Admin

Table 4: Test case bao phủ 5 bước, Lab, cert ngoài và LAN mode.

5.2 Đoạn unit test gợi ý

Unit test nhắm thẳng vào logic của cụm, độc lập hạ tầng socket:

```

1  from core.verify import verify_signature, _dnsname_matches, _validate_fetch_url,
   ↪  UnsafeURLError
2  from cryptography import x509
3
4  def test_trust_store_rong_thi_fail():
5      cert = x509.load_pem_x509_certificate(open("lab/Server-A.crt", "rb").read())
6      ok, msg = verify_signature(cert, trusted_cas=[])      # trust store rỗng
7      assert not ok and "rỗng" in msg
8
9  def test_wildcard_chi_phu_mot_cap():
10     assert _dnsname_matches("*.a.com", "x.a.com") is True
11     assert _dnsname_matches("*.a.com", "y.x.a.com") is False    # 2 cấp -> không khớp
12
13  def test_ssrf_chan_scheme_la():
14      try:
15          _validate_fetch_url("file:///etc/passwd")
16          assert False, "phải raise UnsafeURLError"
17      except UnsafeURLError:
18          pass

```

5.3 Lệnh chạy demo/test

```
# Cài thư viện
pip install -r requirements.txt
pip install pytest

# Chạy bộ scenario end-to-end (5 bước + Lab)
$env:PYTHONIOENCODING = "utf-8"
python tests\test_scenarios.py
python tests\test_m9_external_crl.py      # cert ngoài + CRL

# Mở app + Verification Lab trong Admin Dashboard
python main.py

# Demo LAN: máy A chạy app (bật CSR API), máy B chạy:
python scripts\submit_csr_lan.py --server http://192.168.1.10:8787 ^
  --username alice --password AlicePw123 --domain myshop.com ^
  --san myshop.com,www.myshop.com --token <TOKEN>

# Xuất 1 cert ra PEM để test upload/verify
python scripts\dump_cert.py              # list
python scripts\dump_cert.py 3            # dump cert id=3
```

5.4 Kịch bản demo trực tiếp

1. Mở Verification Lab, thêm 4 server: `valid`, `expired`, `revoked_both`, `tampered`; Verify từng cái, đọc banner PASS/FAIL và log 5 bước.
2. Bấm Renew một server `valid`, Verify lại → pin store báo ROTATION được chấp nhận (cùng issuer).
3. Customer upload một cert (dump bằng `dump_cert.py`), Verify 5 bước.
4. Máy B chạy `submit_csr_lan.py` → Admin refresh CSR Queue thấy request pending → duyệt cấp cert.

6 Tích hợp vào chương trình chung

Cụm này là *mặt phía client* của toàn hệ thống và kết nối với phần CA như sau:

- **Đầu vào cert:** từ socket (`fetch_certificate`), từ DB upload (`external_certs`), hoặc cert do Admin cấp từ CSR gửi qua LAN.
- **Hợp đồng đầu ra** của `verify_certificate_full`: (`overall_pass: bool`, `results: list[(name, ok, msg)], cert`). Mọi UI (Lab GUI, `VerifyExternalDialog`) chỉ cần lấy `results` để in bảng PASS/FAIL và dùng `overall_pass` tô màu banner.

- **Phụ thuộc vào nhóm CA:** Trust Store được nạp từ thư mục mà Admin `publish_active_to_trust_store/publish_root_ca_to_trust_store` ghi ra; CRL/OCSP server (Prod 8889/8888, Lab 9889/9888) cung cấp dữ liệu thu hồi cho B4/B5.
- **LAN mode ghép vào CSR workflow:** `submit_remote_csr` chèn `csr_requests` status `pending` đúng schema, nên CSR Queue UI và `csr_admin` của Admin duyệt như request nội bộ — không cần code riêng cho phê duyệt.

Vì sao đầu ra là list of tuple chứ không chỉ bool? Để UI và test *không* phải gọi lại từng hàm con: chỉ cần lập **results** là có đủ tên bước, trạng thái và thông điệp để hiển thị/assert. Tách dữ liệu (results) khỏi trình bày (UI).

7 Bổ sung sau rà soát: API LAN đầy đủ & chi tiết khác

7.1 LAN không chỉ “nộp CSR” — API customer self-service đầy đủ

Điểm dễ bị hỏi: **chế độ LAN là một API customer self-service hoàn chỉnh** qua HTTP, không phải chỉ `/api/csr/submit`. Khách ở máy B có thể tự theo dõi CSR/cert, gửi & xem yêu cầu thu hồi, và tải CRL — tất cả qua mạng tới máy Admin.

Endpoint (HTTP)	Service (remote_csr)	Ý nghĩa
GET /health	—	Pre-flight: kiểm tra Admin API sống (<code>check_admin_api_health</code>).
POST /api/csr/submit	<code>submit_remote_csr</code>	Nộp CSR (<i>auto-register</i> nếu máy mới).
POST /api/customer/csrs	<code>list_remote_csrs</code>	Liệt kê CSR của chính mình.
POST /api/customer/csr/detail	<code>get_remote_csr_detail</code>	Chi tiết 1 CSR.
POST /api/customer/certs	<code>list_remote_certs</code>	Liệt kê cert đã cấp.
POST /api/customer/cert/detail	<code>get_remote_cert_detail</code>	Chi tiết 1 cert.
POST /api/customer/revoker/submit_request	<code>submit_remote_revocation_request</code>	Gửi yêu cầu thu hồi.
POST /api/customer/revoker/requests	<code>list_remote_revocation_requests</code>	Xem các yêu cầu thu hồi.
POST /api/crl/current	<code>(crl_publish)</code>	Tải CRL hiện hành qua mạng (client: <code>get_crl_from_admin_api</code>).

Table 5: 9 endpoint của Admin API LAN; client gọi qua các hàm `*_from_admin_api` trong `remote_csr_client.py`.

Khác biệt bảo mật then chốt. Chỉ `/api/csr/submit` dùng `_login_or_register_customer`

(*auto-register* cho máy mới lần đầu thuận tiện); *mọi* endpoint còn lại dùng `_login_customer` — **login bắt buộc**, không tạo tài khoản ngoài ý muốn, và bắt buộc `role == "customer"`. Token API + bind loopback (`_is_loopback_bind`) áp dụng cho tất cả.

Ma trận mã trạng thái (`csr_api_server.do_POST`, cap payload 512KB):

Mã	Khi nào
201	Tạo mới thành công (submit CSR / revoke).
200	Đọc thành công (list/detail/CRL).
401	Sai/thiếu token (<code>_authorized</code> fail).
400	JSON hỏng, id sai kiểu, hoặc lỗi nghiệp vụ (<code>RemoteCSRError</code>).
404	Path/đối tượng không tồn tại.
500	Lỗi không lường trước.

7.2 Vài chi tiết verify dễ bị hỏi

- `prune_expired_pins(hostname, base_dir)` (public): ngoài auto-prune bên trong `check_pinned`, còn hàm này dọn pin hết hạn cho một host *theo yêu cầu* (trả về số pin xóa + list fingerprint), dùng bảo trì pin store mà không phải chạy lại 5 bước.
- `check_hostname` còn một nhánh khớp *exact cuối*: sau khi thử IP và wildcard, code còn so trực tiếp `hostname` in `dns_names / ip_addresses` — bắt trường hợp `hostname` là tên DNS thường (không wildcard) khớp đúng một SAN.

8 Checklist chuẩn bị vấn đáp

- ☐ Mở `src/core/verify.py`, chỉ đúng 6 hàm chính: `verify_signature`, `check_validity`, `check_hostname`, `check_crl`, `check_ocsp`, `verify_certificate_full`.
- ☐ Giải thích được từng bước trong 5 bước và vì sao `overall = all(...)`; vì sao pin là advisory.
- ☐ Phân biệt CRL vs OCSP; giải thích scenario `revoked_ocsp_only`.
- ☐ Giải thích guard SSRF/DoS (`_validate_fetch_url`, `_fetch_capped`) và lý do URL trong cert không tin cậy.
- ☐ Giải thích wildcard 1 cấp trong `_dnsname_matches`.
- ☐ Trình bày mô hình 3 trục của `ServerManager` và ý nghĩa từng flavor (đặc biệt `tampered`).
- ☐ Giải thích cert ngoài: dedup theo fingerprint + BOLA guard trong `external_certs`.
- ☐ Giải thích kiến trúc LAN: private key không rời client; token + `_is_loopback_bind` trong `csr_api_server`.
- ☐ Chạy được `python tests\test_scenarios.py` và `test_m9_external_crl.py`.

- ☐ Demo trực tiếp: Lab (valid/expired/revoked/tampered), upload cert verify, gửi CSR qua LAN.

Hết phần phân công chi tiết — Phạm Minh Quân